

N. Shree Deepthi Batch – 37

AI-ASSISTED CODING ASSIGNMENT 5

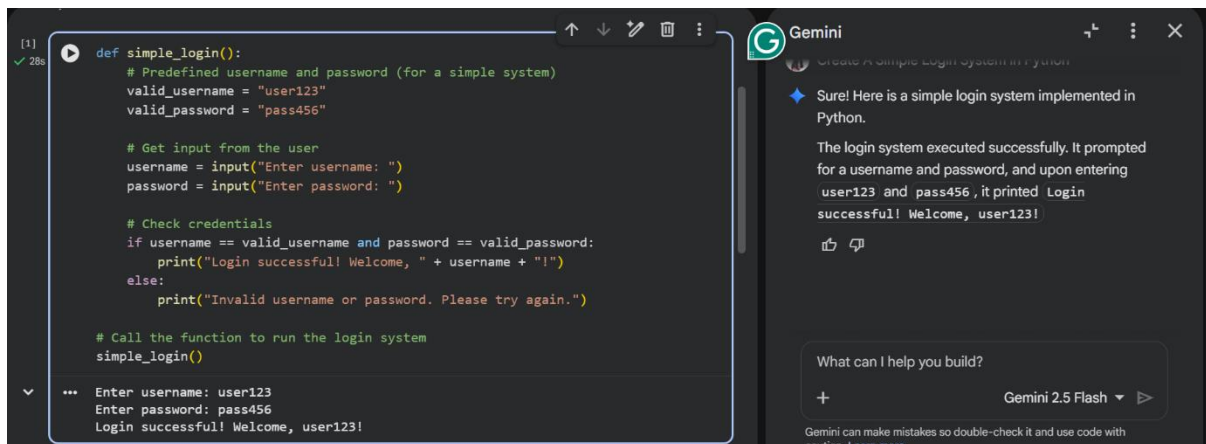
Task – 1 : Privacy and Data Security in AI-Generated Code

Use an AI tool to generate a simple login system in Python. Analyze the generated code to check:

- Whether credentials are hardcoded
- Whether passwords are stored or compared in plain text
- Whether insecure logic is used

Then, revise the code to improve security (e.g., avoid hardcoding, use input validation).

Prompt Used : create a simple login system in python



The screenshot shows a code editor on the left and a Gemini chat window on the right. The code editor displays a Python function `simple_login()` that uses hardcoded credentials. The Gemini chat window shows the AI's response to the prompt 'create a simple login system in python'.

```
[1] def simple_login():  
    # Predefined username and password (for a simple system)  
    valid_username = "user123"  
    valid_password = "pass456"  
  
    # Get input from the user  
    username = input("Enter username: ")  
    password = input("Enter password: ")  
  
    # Check credentials  
    if username == valid_username and password == valid_password:  
        print("Login successful! Welcome, " + username + "!")  
    else:  
        print("Invalid username or password. Please try again.")  
  
    # Call the function to run the login system  
    simple_login()  
  
... Enter username: user123  
Enter password: pass456  
Login successful! Welcome, user123!
```

Gemini

Sure! Here is a simple login system implemented in Python.

The login system executed successfully. It prompted for a username and password, and upon entering user123 and pass456, it printed Login successful! Welcome, user123!

What can I help you build?

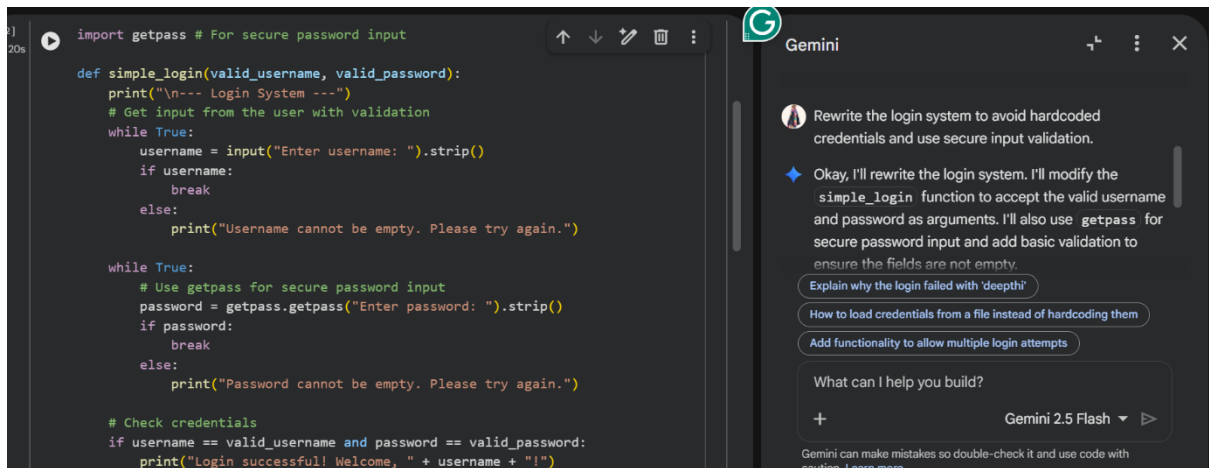
+ Gemini 2.5 Flash ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Analysis : The generated login system uses hardcoded username and password values, which is a major security risk. Passwords are stored in plain text, making them visible to anyone who accesses the code. Users cannot set their own credentials, and sensitive data is not protected.

This violates basic privacy and secure coding practices.

Improved Prompt : Rewrite the login system to avoid hardcoded credentials and use secure input validation.



Explanation : This task highlights security risks in AI-generated authentication code. The original code used hardcoded credentials and plain text password comparison, which is insecure. The revised version improves security by avoiding hardcoding and validating user input.

Task – 2: Bias Detection in AI-Generated Decision Systems

Use AI prompts such as:

- “Create a loan approval system”
- Vary applicant names and genders in prompts

Analyze whether:

- The logic treats certain genders or names unfairly
- Approval decisions depend on irrelevant personal attributes

Suggest methods to reduce or remove bias.

Prompt Used: Create a simple loan approval system in Python based on income and credit score.

[3]
✓ 0s



```
def loan_approval_system(income, credit_score):  
    # Define minimum requirements  
    min_income = 2500 # Minimum required monthly income  
    min_credit_score = 700 # Minimum required credit score  
  
    print(f"\n--- Loan Application Review ---")  
    print(f"Applicant Income: ${income}")  
    print(f"Applicant Credit Score: {credit_score}")  
  
    # Check if requirements are met  
    if income >= min_income and credit_score >= min_credit_score:  
        print("Decision: Approved! Congratulations, your loan application has been approved.")  
    else:  
        print("Decision: Denied. We regret to inform you that your loan application has been denied.")  
        if income < min_income:  
            print(f"Reason: Income is below the minimum required ${min_income}")  
        if credit_score < min_credit_score:  
            print(f"Reason: Credit score is below the minimum required {min_credit_score}")  
  
# Example Usage:  
# Approved case  
print("\n--- Test Case 1: Approved Applicant ---")  
loan_approval_system(income=3000, credit_score=750)
```

Gemini

 Create a simple loan approval system in Python based on income and credit score.

 Certainly! Here's a simple loan approval system in Python. It will consider income and credit score to determine approval.

The loan approval system ran successfully and

Test loan approval with custom inputs

Modify minimum loan approval criteria

Explain the logic of the loan approval system

What can I help you build?

+ Gemini 2.5 Flash ▾ ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```

# Approved case
print("\n--- Test Case 1: Approved Applicant ---")
loan_approval_system(income=3000, credit_score=750)

# Denied case (low income)
print("\n--- Test Case 2: Denied Applicant (Low Income) ---")
loan_approval_system(income=2000, credit_score=720)

# Denied case (low credit score)
print("\n--- Test Case 3: Denied Applicant (Low Credit Score) ---")
loan_approval_system(income=2800, credit_score=680)

# Denied case (both low)
print("\n--- Test Case 4: Denied Applicant (Both Low) ---")
loan_approval_system(income=2000, credit_score=600)

```

```

--- Test Case 1: Approved Applicant ---

--- Loan Application Review ---
Applicant Income: $3000
Applicant Credit Score: 750
Decision: Approved! Congratulations, your loan application has been approved.

--- Test Case 2: Denied Applicant (Low Income) ---

```

```

--- Test Case 2: Denied Applicant (Low Income) ---
...
--- Loan Application Review ---
Applicant Income: $2000
Applicant Credit Score: 720
Decision: Denied. We regret to inform you that your loan application has been denied.
Reason: Income is below the minimum required $2500.

--- Test Case 3: Denied Applicant (Low Credit Score) ---

--- Loan Application Review ---
Applicant Income: $2800
Applicant Credit Score: 680
Decision: Denied. We regret to inform you that your loan application has been denied.
Reason: Credit score is below the minimum required 700.

--- Test Case 4: Denied Applicant (Both Low) ---

--- Loan Application Review ---
Applicant Income: $2000
Applicant Credit Score: 600
Decision: Denied. We regret to inform you that your loan application has been denied.
Reason: Income is below the minimum required $2500.
Reason: Credit score is below the minimum required 700.

```

Explanation: The AI-generated loan approval system makes decisions based only on income and credit score. No personal or sensitive attributes such as gender, name, or age are used in the decision-making process. This ensures that the system does not show bias toward any individual or group. Using only financial factors helps maintain fairness and ethical behavior in AI systems.

Task – 3: Transparency and Explainability in AI-Generated Code (Recursive Binary Search)

Use an AI tool to generate a Python program that:

- Implements Binary Search using recursion
- Searches for a given element in a sorted list
- Includes:
 - o Clear inline comments
 - o A step-by-step explanation of the recursive logic

After generating the code, analyze:

- Whether the explanation clearly describes the base case and recursive case
- Whether the comments correctly match the code logic
- Whether the code is understandable for beginner-level students

Prompt Used : Generate a Python program for recursive binary search. Include clear comments explaining the base case and recursive case.

```
[4] def recursive_binary_search(arr, target, low, high):
✓ 0s # Base Case 1: Element not found
# If the search range (low to high) becomes invalid, the target is not in the array.
if low > high:
    return -1

    mid = (low + high) // 2

# Base Case 2: Element found at mid
# If the element at the middle index is the target, we've found it.
if arr[mid] == target:
    return mid

# Recursive Case 1: Target is in the left half
# If the element at mid is greater than the target, search the left sub-array.
elif arr[mid] > target:
    return recursive_binary_search(arr, target, low, mid - 1)

# Recursive Case 2: Target is in the right half
# If the element at mid is less than the target, search the right sub-array.
else:
    return recursive_binary_search(arr, target, mid + 1, high)

# Example Usage:
my_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
target_found = 23

target_not_found = 10

# Search for an element that exists
index = recursive_binary_search(my_list, target_found, 0, len(my_list) - 1)
if index != -1:
    print(f"Element {target_found} found at index {index}")
else:
    print(f"Element {target_found} not found in the list")

# Search for an element that does not exist
index = recursive_binary_search(my_list, target_not_found, 0, len(my_list) - 1)
if index != -1:
    print(f"Element {target_not_found} found at index {index}")
else:
    print(f"Element {target_not_found} not found in the list")

Element 23 found at index 5
Element 10 not found in the list
```

Gemini

Generate a Python program for recursive binary search. Include clear comments explaining the base case and recursive case.

✦ Certainly! Here's a Python program that implements a recursive binary search.

Explain the time complexity of recursive binary search

How to adapt the binary search for unsorted lists

Test the binary search with an empty list

What can I help you build?

+ Gemini 2.5 Flash ▾ ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Explanation: In this task, the recursive binary search code is easy to understand because clear comments explain what each part does. The base case and recursive calls are clearly mentioned, so it is simple to follow how the algorithm works step by step. This makes the program transparent and helps users trust the logic and results.

Task - 4: Ethical Evaluation of AI-Based Scoring Systems

Ask an AI tool to generate a job applicant scoring system based on features such as:

- Skills
- Experience

- Education

Analyze the generated code to check:

- Whether gender, name, or unrelated features influence scoring
- Whether the logic is fair and objective

Prompt Used: Create a Python program to score job applicants based on skills, experience, and education.

```
def score_applicant(applicant_skills, years_experience, education_level):
    # Define scoring weights for each criterion
    weights = {
        "skills": 5, # Points per matching skill
        "experience": 10, # Points per year of experience (up to a max)
        "education": 15 # Base points for a relevant degree
    }

    # Define ideal skills for the job
    required_skills = ["Python", "SQL", "Data Analysis", "Machine Learning"]

    # Define education level scores
    education_scores = {
        "High School": 0,
        "Associate's Degree": 5,
        "Bachelor's Degree": 15,
        "Master's Degree": 25,
        "PhD": 35
    }

    total_score = 0
    evaluation_details = []
```

Create a Python program to score job applicants based on skills, experience, and education.

◆ Certainly! Here's a Python program to score job applicants based on skills, experience, and education. It defines a scoring function and provides examples of how to use it.

The job applicant scoring system ran successfully, evaluating three example candidates:

What can I help you build?

+ Gemini 2.5 Flash ▾ ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```

# 1. Score for Skills
skill_score = 0
matched_skills = []
for skill in applicant_skills:
    if skill in required_skills:
        skill_score += weights["skills"]
        matched_skills.append(skill)
evaluation_details.append(f"Skills Score: {skill_score} (Matched skills: {'', '.join(matched_skills) if matched_skills else 'None'})")
total_score += skill_score

# 2. Score for Experience
experience_score = min(years_experience, 10) * weights["experience"] # Cap experience at 10 years for scoring
evaluation_details.append(f"Experience Score: {experience_score} ({years_experience} years of experience)")
total_score += experience_score

# 3. Score for Education
education_score = education_scores.get(education_level, 0)
evaluation_details.append(f"Education Score: {education_score} ({education_level})")
total_score += education_score

print("\n--- Applicant Evaluation ---")
print(f"Applicant Skills: {'', '.join(applicant_skills)}")
print(f"Years of Experience: {years_experience}")
print(f"Education Level: {education_level}")
for detail in evaluation_details:
    print(f"    {detail}")
print(f"Total Score: {total_score}")
print(f"-----")

return total_score

# Example Usage:
print("Example Applicant 1: High Score Candidate")
applicant1_score = score_applicant(
    applicant_skills=["Python", "SQL", "Machine Learning", "Communication"],
    years_experience=7,
    education_level="Master's Degree"
)

print("\nExample Applicant 2: Mid-Range Candidate")

```

•• Example Applicant 1: High Score Candidate

```

--- Applicant Evaluation ---
Applicant Skills: Python, SQL, Machine Learning, Communication
Years of Experience: 7
Education Level: Master's Degree
    - Skills Score: 15 (Matched skills: Python, SQL, Machine Learning)
    - Experience Score: 70 (7 years of experience)
    - Education Score: 25 (Master's Degree)
Total Score: 110
-----

```

Example Applicant 2: Mid-Range Candidate

```

--- Applicant Evaluation ---
Applicant Skills: Python, Excel, Data Entry
Years of Experience: 3
Education Level: Bachelor's Degree
    - Skills Score: 5 (Matched skills: Python)
    - Experience Score: 30 (3 years of experience)
    - Education Score: 15 (Bachelor's Degree)
Total Score: 50
-----

```

Example Applicant 3: Entry-Level Candidate

```

--- Applicant Evaluation ---
Applicant Skills: Customer Service
Years of Experience: 1
Education Level: High School
    - Skills Score: 0 (Matched skills: None)
    - Experience Score: 10 (1 years of experience)
    - Education Score: 0 (High School)
Total Score: 10

```


Explanation : In this task, the job applicant scoring system evaluates candidates based only on skills, experience, and education, which are directly related to job performance. The program does not use personal details like name, gender, or age, so it avoids unfair bias. Since the scoring criteria are clearly defined and job-relevant, the system follows ethical principles and promotes fair decision-making.

Task - 5: Inclusiveness and Ethical Variable Design

Use an AI tool to generate a Python code snippet that processes user or employee details.

Analyze the code to identify:

- Gender-specific variables (e.g., male, female)
- Assumptions based on gender or identity
- Non-inclusive naming or logic

Modify or regenerate the code to:

- Use gender-neutral variable names
- Avoid gender-based conditions unless strictly required
- Ensure inclusive and respectful coding practices

Prompt Used: Create a Python program that processes employee details such as name, age, gender, and salary.

The screenshot displays the Gemini AI interface. On the left, a Python code snippet is shown, defining an `Employee` class and an `EmployeeManagementSystem` class. The `Employee` class has attributes `name`, `age`, `gender`, and `salary`. The `EmployeeManagementSystem` class has a list `employees` and methods to add employees, display all employees, and calculate the average salary. The code includes example usage at the bottom.

```
class Employee:
    def __init__(self, name, age, gender, salary):
        self.name = name
        self.age = age
        self.gender = gender
        self.salary = salary

    def display_employee(self):
        print(f"Name: {self.name}, Age: {self.age}, Gender: {self.gender}, Salary: ${self.salary:.2f}")

class EmployeeManagementSystem:
    def __init__(self):
        self.employees = []

    def add_employee(self, name, age, gender, salary):
        employee = Employee(name, age, gender, salary)
        self.employees.append(employee)
        print(f"Employee {name} added successfully.")

    def display_all_employees(self):
        if not self.employees:
            print("No employees in the system.")
            return
        print("\n--- All Employees ---")
        for emp in self.employees:
            emp.display_employee()
        print("-----")

    def calculate_average_salary(self):
        if not self.employees:
            return 0
        total_salary = sum(emp.salary for emp in self.employees)
        return total_salary / len(self.employees)

# Example Usage:
emp_system = EmployeeManagementSystem()

emp_system.add_employee("Alice Smith", 30, "Female", 60000)
emp_system.add_employee("Bob Johnson", 35, "Male", 75000)
emp_system.add_employee("Charlie Brown", 28, "Male", 55000)
```

On the right, the Gemini AI interface shows the prompt: "Create a Python program that processes employee details such as name, age, gender, and salary." The AI response includes a confirmation and a summary of the program's functionality, mentioning that it added three employees (Alice Smith, Bob Johnson, and Charlie Brown) and calculated the average salary (\$63333.33). Below the response, there are interactive buttons: "Add a function to search employees by name", "Filter employees earning more than \$70000", and "Explain the Employee class in the last code cell". At the bottom, there is a section titled "What can I help you build?" with a search bar and a "Gemini 2.5 Flash" button.

```
emp_system.display_all_employees()

average_salary = emp_system.calculate_average_salary()
print(f"\nAverage Salary of Employees: ${average_salary:.2f}")
```

```
"" Employee Alice Smith added successfully.
Employee Bob Johnson added successfully.
Employee Charlie Brown added successfully.

--- All Employees ---
Name: Alice Smith, Age: 30, Gender: Female, Salary: $60000.00
Name: Bob Johnson, Age: 35, Gender: Male, Salary: $75000.00
Name: Charlie Brown, Age: 28, Gender: Male, Salary: $55000.00
-----

Average Salary of Employees: $63333.33
```

Explanation : In this task, the employee management system processes details like name, age, gender, and salary without making decisions based on gender. Although gender information is collected, it is not used to affect salary or employee handling, which helps avoid bias. The program treats all employees equally and uses neutral logic, making it more inclusive and ethically designed.